TransitionFlowDiagram UI設計書

メタデータ

- **コンポーネントID**: TransitionFlowDiagram
- **コンポーネント名**: トランジションフロー図
- **ファイルパス**: `frontend/src/components/ExperimentWorker/TransitionFlowDiagram.tsx`
- **利用権限**: 実験作業者（experiment_worker）
- **作成日**: 2026-01-19
- **最終更新日**: 2026-03-11

---

1. 概要

1.1 コンポーネントの目的

選択されたメソッドに含まれるトランジションをReactFlowを使用してフロー図として視覚的に表示し、実験の流れを把握しやすくするためのコンポーネントです。

1.2 主要機能

- トランジションのフロー図表示（React Flow）
- workOrder順での自動レイアウト
- ノードのドラッグ移動
- ズーム・パン操作
- ミニマップ表示
- ステータスの色分け表示
- 各ノードから記録・問題報告へのアクセス

1.3 使用場所

- ExperimentRecordTab（実験記録タブ）のフロー図表示モード

---

2. コンポーネント構成

2.1 画面イメージ


トランジションフロー図イメージ

2.2 レイアウト構造

```
┌─────────────────────────────────────────────┐
│ ReactFlow コンテナ (500px高さ)              │
│ ┌─────────────────────────────────────────┐ │
│ │ Panel (top-left)                        │ │
│ │ ┌──────────────┐                        │ │
│ │ │ メソッド名   │                        │ │
│ │ │ トランジション数│                     │ │
│ │ │ 操作ヒント   │                        │ │
│ │ └──────────────┘                        │ │
│ │                                         │ │
│ │  [Node1] → [Node2] → [Node3]            │ │
│ │     ↓                                   │ │
│ │  [Node4] → [Node5] → [Node6]            │ │
│ │     ↓                                   │ │
│ │  [Node7] → [Node8]                      │ │
│ │                                         │ │
│ │ ┌──────────────┐ ┌──────────────┐       │ │
│ │ │ Controls     │ │ MiniMap      │       │ │
│ │ │ (ズーム・フィット)│ │ (全体マップ) │       │ │
│ │ └──────────────┘ └──────────────┘       │ │
│ │                                         │ │
│ │ Background (グリッド)                   │ │
│ └─────────────────────────────────────────┘ │
└─────────────────────────────────────────────┘

空状態:
┌─────────────────────────────────────────────┐
│ Empty                                       │
│  [PartitionOutlined アイコン (48px)]        │
│  トランジションがありません                 │
│  選択されたタスクにトランジションが         │
```

```
│  含まれていません                                    │
└──────────────────────────────────────────────────┘
```

2.3 使用コンポーネント

React Flow コンポーネント

- `ReactFlow` - メインフローコンテナ
- `Background` - 背景グリッド
- `Controls` - ズーム・フィットコントロール
- `MiniMap` - ミニマップ
- `Panel` - 情報パネル
- `useNodesState` - ノード状態管理フック
- `useEdgesState` - エッジ状態管理フック
- `ConnectionMode` - 接続モード設定
- `MarkerType` - エッジマーカータイプ

Ant Design コンポーネント

- `Empty` - 空状態表示
- `Space` - 要素の間隔制御
- `Typography.Text` - テキスト表示
- `Typography.Title` - タイトル表示
- `theme` - デザイントークン取得

カスタムコンポーネント

- `TransitionFlowNode` - トランジションノード（カスタムノードタイプ）

アイコン

- `PartitionOutlined` - フロー図アイコン（空状態）

---

3. Props

```
interface TransitionFlowDiagramProps {
  transitions: Transition[];              // トランジション一覧
  onRecordClick: (transition: Transition) => void; // 記録ボタンクリックハンドラ
  onIssueClick: (transition: Transition) => void;  // 問題報告ボタンクリックハンドラ
  isProjectRecordable: boolean;           // プロジェクトの記録可能フラグ
  methodName?: string;                    // メソッド名（情報パネルに表示）
}
```

---

4. ノードとエッジの生成

4.1 ノード自動レイアウト

```
// レイアウトパラメータ
const horizontalSpacing = 270; // 横間隔
const verticalSpacing = 180;   // 縦間隔
const nodesPerRow = 4;         // 1行あたりのノード数

// 位置計算
const row = Math.floor(index / nodesPerRow);
const col = index % nodesPerRow;

position: {
  x: col * horizontalSpacing + 50,
  y: row * verticalSpacing + 50,
}
```

レイアウト例（12ノード）：

```
[1] → [2] → [3] → [4]
 ↓
[5] → [6] → [7] → [8]
```

```
 ↓
[9] → [10] → [11] → [12]
```

4.2 ノード定義

```
{
  id: transition.transitionId,
  type: 'transitionNode',        // カスタムノードタイプ
  position: { x, y },
  data: {
    transition,                   // トランジションデータ
    onRecordClick,                // 記録ハンドラ
    onIssueClick,                 // 問題報告ハンドラ
    isProjectRecordable,          // 記録可能フラグ
  },
  draggable: true,                // ドラッグ可能
}
```

4.3 エッジ定義

workOrder順に隣接するノード間を接続：

```
{
  id: `edge-${current.transitionId}-${next.transitionId}`,
  source: current.transitionId,
  target: next.transitionId,
  type: 'smoothstep',             // スムーズなステップ線
  animated: false,                // 実線（アニメーションなし）
  style: {
    stroke: '#000000',            // 黒色
    strokeWidth: 2,
  },
  markerEnd: {
    type: MarkerType.ArrowClosed,
    color: '#000000',
    width: 20,
    height: 20,
  },
}
```

---

5. ReactFlow設定

5.1 ビューポート設定

```
fitView
fitViewOptions={{
  padding: 0.2,      // パディング20%
  minZoom: 0.5,      // 最小ズーム50%
  maxZoom: 1.5,      // 最大ズーム150%
}}
minZoom={0.3}        // 全体最小ズーム30%
maxZoom={2}          // 全体最大ズーム200%
defaultViewport={{ x: 0, y: 0, zoom: 1 }}
```

5.2 操作設定

```
nodesDraggable={true}          // ノードドラッグ可能
nodesConnectable={false}       // ノード接続無効
elementsSelectable={true}      // 要素選択可能
panOnDrag={true}               // ドラッグでパン
panOnScroll={false}            // スクロールではパンしない
zoomOnScroll={true}            // スクロールでズーム
zoomOnPinch={true}             // ピンチでズーム
zoomOnDoubleClick={false}      // ダブルクリックでズーム無効
connectionMode={ConnectionMode.Loose}
```

---

6. コントロール要素

6.1 Background（背景グリッド）

```
<Background
  color={token.colorBorder}
  gap={20}                     // グリッド間隔20px
```

```
  size={1}                    // 線の太さ1px
/>
```

6.2 Controls（コントロールパネル）

標準のReact Flowコントロール：

- ズームイン
- ズームアウト
- フィットビュー
- インタラクティブモード切替（無効化）

```
<Controls showInteractive={false} />
```

6.3 MiniMap（ミニマップ）

ノードの色をステータスに応じて変更：

ステータス	色
not_started	colorBorder（グレー）
in_progress	colorWarning（オレンジ）
completed	colorSuccess（緑）
on_hold	colorWarning（オレンジ）
issue	colorError（赤）

```
<MiniMap
  nodeColor={(node) => {
    // ステータスに応じた色を返す
  }}
  maskColor={`${token.colorBgContainer}CC`}
  style={{
    background: token.colorBgContainer,
    border: `1px solid ${token.colorBorder}`,
    borderRadius: token.borderRadiusSM,
```

```
  }}
/>
```

6.4 Panel（情報パネル）

左上に配置される情報表示パネル：

- メソッド名（props経由）
- トランジション数
- 操作ヒント（"ドラッグで移動、スクロールでズーム"）

```
<Panel position="top-left">
  <div style={{ /* スタイル */ }}>
    <Space orientation="vertical" size="small">
      {methodName && (
        <Text strong style={{ fontSize: 14 }}>
          {methodName}
        </Text>
      )}
      <Text type="secondary" style={{ fontSize: 12 }}>
        トランジション数: {transitions.length}
      </Text>
      <Text type="secondary" style={{ fontSize: 11 }}>
        ドラッグで移動、スクロールでズーム
      </Text>
    </Space>
  </div>
</Panel>
```

---

7. イベント

7.1 ノード・エッジ変更イベント

React Flowの標準イベント：

```
onNodesChange={onNodesChange}  // ノード位置変更時
```

```
onEdgesChange={onEdgesChange}  // エッジ変更時（現在は発生しない）
```

7.2 カスタムノード内のイベント

TransitionFlowNode内で定義：

イベント	トリガー	処理内容
onRecordClick	記録ボタンクリック	props経由で親コンポーネントに通知
onIssueClick	問題報告ボタンクリック	props経由で親コンポーネントに通知

---

8. データフロー

8.1 State（内部状態）

```
// React Flow標準のフック
const [nodes, setNodes, onNodesChange] = useNodesState(initialNodes);
const [edges, setEdges, onEdgesChange] = useEdgesState(initialEdges);
```

8.2 算出データ（useMemo）

```
// 初期ノード生成（workOrder順にソート、自動レイアウト）
const initialNodes = useMemo(() => {
  // workOrder順にソート
  const sortedTransitions = [...transitions].sort((a, b) => {
    if (a.workOrder !== undefined && b.workOrder !== undefined) {
      return a.workOrder - b.workOrder;
    }
    return 0;
  });

  // ノード生成（レイアウト計算）
  // ...
}, [transitions, onRecordClick, onIssueClick, isProjectRecordable]);

// 初期エッジ生成（workOrder順の隣接接続）
```

```
// ※ token.colorPrimary は依存配列に含まれているが、内部では使用していない（過剰な依存）
const initialEdges = useMemo(() => {
  // workOrder順にソート
  // 隣接トランジション間にエッジ追加
  // ...
}, [transitions, token.colorPrimary]);
```

8.3 useEffectによる同期

```
// ノードデータが変更された場合に更新
React.useEffect(() => {
  setNodes(initialNodes);
}, [initialNodes, setNodes]);

React.useEffect(() => {
  setEdges(initialEdges);
}, [initialEdges, setEdges]);
```

---

9. スタイリング

9.1 コンテナスタイル

```
{
  width: '100%',
  height: '500px',
  border: `1px solid ${token.colorBorder}`,
  borderRadius: token.borderRadiusLG,
  overflow: 'hidden',
  background: token.colorBgLayout,
}
```

9.2 空状態スタイル

```
{
  width: '100%',
  height: '500px',
  display: 'flex',
  alignItems: 'center',
  justifyContent: 'center',
  background: token.colorBgLayout,
  borderRadius: token.borderRadiusLG,
}
```

9.3 情報パネルスタイル

```
{
  background: token.colorBgContainer,
  padding: `${token.paddingSM}px ${token.paddingMD}px`,
  borderRadius: token.borderRadiusSM,
  border: `1px solid ${token.colorBorder}`,
  boxShadow: token.boxShadow,
}
```

---

10. 空状態処理

10.1 表示条件

```
if (!transitions || transitions.length === 0) {
  return (
    <div
      style={{
        width: '100%',
        height: '500px',
        display: 'flex',
        alignItems: 'center',
        justifyContent: 'center',
        background: token.colorBgLayout,
        borderRadius: token.borderRadiusLG,
      }}
    >
```

```
      <Empty
        image={<PartitionOutlined style={{ fontSize: 48, color: token.colorTextDisabled }} />}
        imageStyle={{ height: 48 }}
        description={
          <Space orientation="vertical" align="center" size="small">
            <Title level={4} type="secondary" style={{ margin: 0 }}>
              トランジションがありません
            </Title>
            <Text type="secondary">
              選択されたタスクにトランジションが含まれていません
            </Text>
          </Space>
        }
      />
    </div>
  );
}
```

---

11. パフォーマンス考慮事項

11.1 最適化

- `initialNodes` と `initialEdges` は `useMemo` でメモ化
- React Flowの標準最適化機能を活用
- ノードの再レンダリングを最小化

11.2 データ量

- **想定ノード数**: 最大50件程度
- **1行あたりのノード数**: 4件
- **最大行数**: 13行程度（50 / 4）

---

12. 備考

12.1 特記事項

1. **workOrderによる自動ソート**
    - トランジションはworkOrder順にソートして表示
    - workOrderがない場合は元の順序を維持
2. **自動レイアウト**
    - 左から右、上から下の順で配置
    - 4列グリッドレイアウト
3. **ノード間の接続**
    - workOrder順の隣接トランジション間を黒色実線で接続
    - すべての隣接ペアを接続（アニメーションなし）
4. **カスタムノードタイプ**
    - TransitionFlowNodeを使用
    - ノード内にトランジション情報とボタンを配置
5. **操作性**
    - ノードはドラッグ移動可能
    - スクロールでズーム
    - ドラッグでパン
    - ノード間の新規接続は無効化

12.2 将来の拡張

1. レイアウトアルゴリズムの改善（階層レイアウト、DAGレイアウト）
2. ノード間の依存関係の可視化
3. フロー図のエクスポート機能（PNG, SVG）
4. ノードのフィルタリング機能
5. トランジションの詳細情報ポップオーバー
6. パスの強調表示（開始→完了のルート）

12.3 関連ドキュメント

- ExperimentRecordTab UI設計書
- TransitionFlowNode UI設計書
- ReactFlow 公式ドキュメント

---

13. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-01-19	初版作成	Cline
1.0.1	2026-03-11	ソースコードとの差分修正（Space.orientation化、空状態divラッパー追加、アイコンスタイル追加、Textのfontsize追加、useMemo依存配列の補足、ReactFlowインポート追記）	Cline